

BAN CƠ YẾU CHÍNH PHỦ
HỌC VIỆN KỸ THUẬT MẬT MÃ



LẬP TRÌNH NHÂN LINUX PHẦN 2. TỔNG QUAN

Ngành: Công nghệ thông tin

Chuyên ngành: Kỹ thuật phần
mềm nhúng và di động

Mã số: 52.48.02.01

Nội dung

- Introduction
- Device Tree syntax
- Use device in driver
- Practice

Introduction

Old style of kernel code

```
1 #define GPIO_BASE_ADDR      0x100000
2 #define GPIO1_OFFSET        0x4
3 #define GPIO2_OFFSET        0x8
4 #define GPIO_IRQ_NUM        0x2
5
6
7 __writel(0x2, GPIO_BASE_ADDR + GPIO1_OFFSET);
8 __writel(0x4, GPIO_BASE_ADDR + GPIO2_OFFSET);
9 regist_irq(GPIO_IRQ_NUM);
```

New style with device tree

Diagram illustrating the new style with device tree, showing a node structure:

```
node {
    label: serial@40027000 {
        node name
        compatible = "fsl,vf610-lpuart";
        reg = <0x40027000 0x1000>;
        property
        clocks = <&clks VF610 CLK UART0>;
        clock-names = "ipg";
        interrupts = <GIC_SPI 61 IRQ_TYPE_LEVEL_HIGH>;
        dmas = <&edma0 0 2>,
              <&edma0 0 3>;
        dma-names = "rx", "tx";
        status = "disabled";
    };
}
```

```
11 reg = platform_get_resource(pdev, IORESOURCE_MEM, 0);
12 reg_base = devm_ioremap_resource(&pdev->dev, reg);
13 __writel(0x4, reg_base + GPIO2_OFFSET);
14 s->irq = platform_get_irq(pdev, 0);
```

Device tree advantage

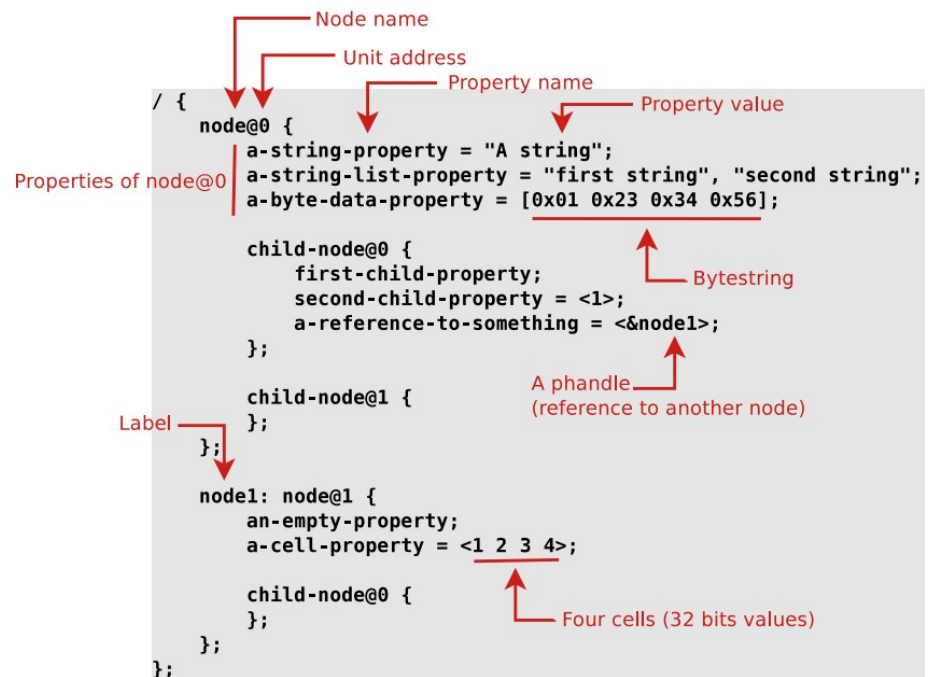
- Simple to change the configuration
- Easily add support for new hardware
- Can reuse and over ride existing .dts files
- Easy to read and understand descriptions of hardware

Device tree building

- Các file dtsi
 - Chứa các thông tin chung của SoC
- Các file dts
 - Chứa thông tin private của board
- Các file dtb
 - `make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- make dtbs`
 - `dtc -I dtb -O dtsarch/arm/boot/dts imx6dl-sabrelite.dtb >path/to/my_devicetree.dts`

Device Tree Basic syntax

- Device node
- Label
- Quy ước đặt tên
- Quy ước đặt giá trị
- Số nguyên 32 bit và mảng



The diagram illustrates the Device Tree Basic syntax with annotations for various components:

- Node name:** Points to `node@0` in `node@0 {`.
- Unit address:** Points to `0` in `node@0`.
- Property name:** Points to `a-string-property`.
- Property value:** Points to `"A string"`.
- Properties of node@0:** Points to the list of properties: `a-string-property`, `a-string-list-property`, and `a-byte-data-property`.
- Bytestring:** Points to the byte data value `[0x01 0x23 0x34 0x56]`.
- A phandle (reference to another node):** Points to `&node1` in `a-reference-to-something = <&node1>;`.
- Label:** Points to the label `node1:` in `node1: node@1 {`.
- Four cells (32 bits values):** Points to the cell array `<1 2 3 4>` in `a-cell-property = <1 2 3 4>;`.

```
/ {  
    node@0 {  
        a-string-property = "A string";  
        a-string-list-property = "first string", "second string";  
        a-byte-data-property = [0x01 0x23 0x34 0x56];  
  
        child-node@0 {  
            first-child-property;  
            second-child-property = <1>;  
            a-reference-to-something = <&node1>;  
        };  
  
        child-node@1 {  
        };  
    };  
  
    node1: node@1 {  
        an-empty-property;  
        a-cell-property = <1 2 3 4>;  
  
        child-node@0 {  
        };  
    };  
};
```

Đăng ký và sử dụng device node

- Đa số sử dụng kiểu of match table
 - Driver đăng ký bảng of match table có chứa các chuỗi compatible
 - Device node có trường compatible
 - Nếu 2 chuỗi compatible trùng nhau thì hàm probe của driver được gọi và truyền vào con trỏ chứa device node

Các template information của device tree

- Các cú pháp chuẩn
- Các cú pháp tự định nghĩa

Các cú pháp chuẩn

- Định địa chỉ bus
- Định nghĩa ngắt
- <https://events.static.linuxfound.org/sites/events/files/slides/petazzoni-device-tree-dummies.pdf>
- `/include/linux/of.h`

Các cú pháp tự định nghĩa

- Được get ra bởi các hàm parser array, value
 - `one-int-property = <197>; int-list-property = <1350000 0x54dae47 1250000 1200000>;`
 - `int of_property_read_u32_array(const struct device_node *np, const char *propname, u32 *out_values, size_t sz);`
- Driver phải tự xử lý thông tin từ dữ liệu thô

Các cú pháp hỗ trợ bởi driver code

- Các hàm parser data nằm phân tán ở driver core
- Tìm hiểu qua `/Documentation/devicetree/bindings`
- Example

Kết hợp giữa device tree và code C để define phần cứng

- 1 driver điều khiển nhiều device gần giống nhau về hardware nhưng có chức năng khác nhau. Ví dụ như:
 - 2 đèn led có tần số nhấp nháy khác nhau.
 - Port Ethernet support các mode 10M, 100M, 1G.

Kết hợp giữa device tree và code C để define phần cứng

- Define struct chứa các thông số config phần cứng.

```
19 struct led_config_t {  
20     uint32_t led;  
21     uint32_t data_out;  
22     uint32_t time;  
23 };  
24
```

Kết hợp giữa device tree và code C để define phần cứng

- Gán các thông số cho struct config phần cứng.

```
32 struct led_config_t led_gpio30_config = {  
33     .led = GPIO30,  
34     .data_out = GPIO30_DATA_OUT,  
35     .time = 1,  
36 };
```

Kết hợp giữa device tree và code C để define phần cứng

- Đính kèm struct chứa config của phần cứng vào node device tree.

```
56 static const struct of_device_id blink_led_of_match[] = {  
57     { .compatible = "led-example1", .data = &led_gpio30_config},  
58     { .compatible = "led-example2", .data = &led_gpio03_config},  
59     {},  
60 };
```

```
of_id = of_match_device(blink_led_of_match, &pdev->dev);  
timer_data->led_config = of_id->data;
```

Kết hợp giữa device tree và code C để define phần cứng

- Define struct chứa các dữ liệu đặc trưng cho kernel (driver data).

```
25 struct led_driver_data {  
26     void __iomem *base_addr;  
27     unsigned int count;  
28     const struct led_config_t *led_config;  
29     struct timer_list my_timer;  
30 };
```


Kết hợp giữa device tree và code C để define phần cứng

- Đính kèm driver data cho timer function.

```
timer_data->my_timer.data = (unsigned long)timer_data;
```

```
44 static void timer_function(unsigned long data){  
45     struct led_driver_data *timer_data = (struct led_driver_data *)data;  
46  
47     if ((timer_data->count % 2) == 0)  
48     |     writel_relaxed(timer_data->led_config->data_out, timer_data->base_addr  
49     | )
```

Booting flow

- Boot loader load kernel image and dtb file
- Initialize hardware data base
- Call probe function of all drives
- `setup_arch(&command_line)` ->
- `setup_machine_fdt(__atags_pointer)` ->
- `unflatten_device_tree()`.

HỎI - ĐÁP
